

AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING



Specialized Program – Junior Level

Computer and systems Engineering Department – Spring 26

CSE472s: Artificial Intelligence

Quoridor Game Implementation

Name	ID
Amr Ahmed Nagy	2300162
Mohamed Ashraf Mohamed	2300475
Omar Ahmed Morshed	2300131
Ali Ahmed Ayman	2300346
Mohamed Ehab Abdelbary	2300570

The repo link: <https://github.com/amrrnagy/Quoridor>

The demo video link: <https://www.youtube.com/watch?v=WKwINbpfrHU>

Submitted to:

Dr: Manal Morad Zaki

Eng: Robeir Samir

1.Introduction:

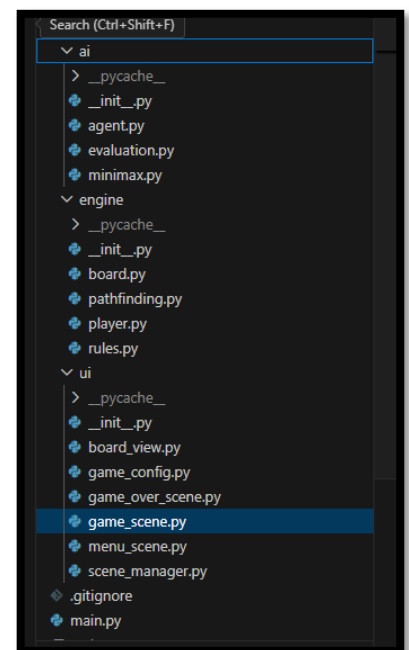
The project encompasses a complete game engine enforcing all official Quoridor rules, a graphical user interface (GUI) built with Pygame, and an AI opponent implemented via the Minimax algorithm with Alpha-Beta pruning.

Quoridor, designed by Mirko Marchesi and first published in 1997, is an abstract strategy game played on a 9×9 grid in which two players race to be the first to move their pawn to the opposite baseline while strategically placing walls to obstruct the opponent. Its combination of movement planning and constraint placement provides a rich environment for exploring classical AI search techniques.

2.Design Decisions and Architecture:

2.1 Architectural Overview

The project adopts a strict three-layer architecture that enforces separation of concerns across the game engine, the AI subsystem, and the user interface. This design follows the Model-View-Controller (MVC) pattern, ensuring that each layer can be developed, tested, and modified independently.



Layer	Module(s)	Responsibility
Game Engine	engine/board.py, rules.py, pathfinding.py, player.py	Board state, rule enforcement, wall validation, pathfinding
AI Subsystem	ai/minimax.py, ai/evaluation.py, ai/agent.py	Move generation, heuristic scoring, difficulty dispatch
User Interface	ui/board_view.py, game_scene.py, menu_scene.py, scene_manager.py	Rendering, input handling, scene transitions

2.2 Game Engine Layer

The engine layer is entirely free of rendering or AI logic. The Board class maintains a 9×9 grid of cell objects, two separate sets of horizontal and vertical wall segment objects, and the current game-state metadata (active player, move history). This separation ensures that the AI can query and clone the board state without triggering any graphical side effects.

The Rules module exposes a single public function, `get_legal_moves(board, player)`, which returns the complete set of valid actions (pawn moves and wall placements) for the given player. Jump logic including straight jumps over adjacent pawns and diagonal sidestep jumps blocked by walls or board edges is fully encapsulated here.

2.3 Pathfinding and wall Validation:

A critical correctness requirement is that no wall placement may completely block a player's path to their goal row. The Pathfinding module implements a Breadth-First Search (BFS) from each player's current position to any cell in the opponent's goal row. Before a wall placement is accepted, the engine provisionally applies the wall to a cloned board and verifies that a valid path still exists for both players. If either player's path is eliminated, the placement is rejected as illegal.

BFS was preferred over Dijkstra or A* for path existence checks because it runs in $O(V + E)$ time over the small 9×9 graph ($V = 81$, $E \leq 288$), making it computationally negligible even when called hundreds of times per AI search tree evaluation.

2.4 User interface layer:

The UI layer is built on Pygame and managed by a SceneManager that switches between the MainMenu, GameScene, and GameOverScene without restarting the application. All rendering constants (board size, cell dimensions, colours, fonts) are centralised in `game_config.py`, so visual adjustments require changes in only one location.

The GameScene handles all user interaction for the current game turn: pawn selection and highlighting, wall preview rendering, illegal-move feedback and real-time wall-count display. The design intentionally prevents users from submitting illegal moves rather than merely displaying post-hoc error messages.

3. Implementation challenges and solutions

3.1 Wall Representation and Edge Indexing

Challenge: Walls in Quoridor lie on the edges between cells rather than on cells themselves, making intuitive indexing non-trivial. A 9×9 board produces an 8×9 grid of possible horizontal wall positions and a 9×8 grid of vertical wall positions.

Solution: We represent each wall as a (row, col, orientation) triple, where the coordinates identify the top-left cell of the two-cell span. Separate boolean arrays `h_walls[8][8]` and `v_walls[8][8]` store placed walls. Collision detection checks whether a new wall would share any segment with or cross perpendicularly through an existing wall, a condition expressed as a simple set of index arithmetic comparisons.

3.2 Diagonal Jump Rules

Challenge: The official Quoridor jump rules are nuanced: a player may jump over an adjacent opponent pawn in a straight line unless a wall blocks the path or the board edge is reached, in which case the player may jump to either diagonal cell adjacent to the opponent. Implementing all edge cases correctly (double walls, board corners, multiple blocking conditions) proved to be the most rule-intensive aspect of the project.

Solution: We decomposed the jump logic into a decision tree. The Rules module first checks whether a straight jump is geometrically valid (within bounds) and unobstructed. If not, it iterates over the two diagonal offsets and verifies that each candidate cell is within bounds and reachable without crossing a wall. This decomposition made the logic independently unit-testable.

3.3 AI Search Performance:

Challenge: A naïve Minimax search over the full Quoridor game tree is computationally intractable. The branching factor at each node is up to 140 and games can last 50–100 turns.

Solution: Three optimisations were applied: (1) Alpha-Beta pruning, which eliminates branches provably worse than a known alternative, reducing the effective branching factor by approximately 50%. (2) A depth limit of 3 plies for the Hard difficulty, which was empirically determined to produce strong play within a 2-second response time budget on a standard laptop. (3) Move ordering sorting candidate moves by the heuristic score before expansion to maximise the pruning benefit of Alpha-Beta.

3.4 Undo/Redo State Management

Challenge: Implementing undo/redo requires preserving complete, immutable snapshots of game state at every turn, including board configuration, active player, wall counts, and move history.

Solution: The GameScene maintains two stacks: an `undo_stack` that receives a deep copy of the Board object before every action is applied, and a `redo_stack` that receives the current state when an undo is performed. Redo is invalidated on any new move. This approach leverages `board.copy()`, already required for AI tree search, at no additional development cost.

4. AI Algorithm Explanation:

4.1 Overview

The AI subsystem is organized across three modules: `agent.py` configures difficulty parameters and owns the position-history memory; `minimax.py` implements Iterative Deepening Minimax with Alpha-Beta pruning and a Transposition Table; and `evaluation.py` implements the heuristic scoring function. This separation allows difficulty tuning and heuristic adjustment without touching the core search logic.

4.2 AI Agent: Difficulty Configuration and Entry Point

The AI Agent class (`agent.py`) serves as the single public API for the game scene. It selects search parameters based on the chosen difficulty and maintains a rolling window of the AI's recent positions to feed the loop-killer penalty.

The `depth` parameter sets the maximum search depth for Iterative Deepening. The `adv` flag enables the advanced heuristic components (wall advantage, tempo, centralization, proximity) in `evaluation.py`. The `time` parameter is the per-move wall-clock budget in seconds.

Before delegating to the search, `get_best_move()` appends the current position to `self.position_history` and trims the list to the six most recent entries, preventing unbounded memory growth while still covering common oscillation patterns.

4.3 Iterative Deepening with Time Management

`get_best_move_iterative()` in `minimax.py` performs depth-first searches from depth 1 up to `max_depth`, returning the result of the last fully completed iteration:

The 10% budget guard prevents launching a deep iteration with insufficient time remaining, which would be discarded anyway. The transposition table is cleared at the start of each move so stale entries from the previous turn do not pollute the new search.

4.4 Minmax with Alpha-Beta Pruning and Transposition Table

The `minimax()` function implements standard two-player Minimax with Alpha-Beta pruning. A transposition table (a Python dict keyed on a tuple of both pawns' positions, frozen wall sets, and the current player) stores (depth, score, best_move) entries. A cached entry is only used when its stored depth is greater than or equal to the requested depth, ensuring correctness:

Alpha-Beta cutoffs are applied in the standard way: at a maximizing node, a branch is pruned when its value exceeds beta; at a minimizing node, when it falls below alpha. This can reduce the effective number of nodes evaluated from $O(b^d)$ to $O(b^{(d/2)})$ under ideal move ordering.

4.5 Strategic Move Ordering

Move ordering is critical to Alpha-Beta efficiency. `_get_strategic_actions()` constructs a Hot Zone of tactically relevant board cells and uses it to filter and rank wall candidates before the search begins. The Hot Zone is built from three layers:

- Layer A: All cells within a radius-2 Manhattan distance of any cell on either player's BFS shortest path. This captures walls that force one- or two-cell detours, the most common tactical interventions.
- Layer B: A 2-cell halo around each pawn, ensuring threats immediately adjacent to a player are never ignored regardless of path structure.
- Layer C (implicit): Walls near the path intersection of both players, where a single wall can simultaneously lengthen one path and shorten another.

Each surviving wall candidate is then assigned a priority score by `_score_wall()`, which estimates the detour it imposes on the opponent and the self-harm it causes:

Pawn moves are always included and sorted so goal-advancing moves (lower distance to goal row) are tried first, ensuring the most likely best move is evaluated earliest and produces tight alpha-beta bounds for subsequent branches.

4.6 Heuristic Evaluation Function

`evaluate_board()` in `evaluation.py` returns a score from the AI's perspective. The function first checks for terminal states ($\pm 10,000$), then computes path lengths via BFS. The base score is the path-length difference (opponent distance minus AI distance), amplified by an end-game multiplier when the opponent is within two steps of their goal:

When the advanced heuristic is active (Hard difficulty), four additional weighted components are added

4.7 Difficulty Level Summary

Level	Max Depth	Time Budget	Advanced Heuristic	Characteristics
Easy	1	0.5 s	No	Examines only immediate actions; no lookahead. Suitable for beginners learning the rules.
Medium	6	2.0 s	No	Iterative deepening to depth 6 with base heuristic (path difference only). Plays solid positional moves but does not exploit wall tempo.
Hard	10	4.5 s	Yes	Full search with advanced heuristic, Hot-Zone wall filtering, scored move ordering, and loop-killer penalty. Strong amateur-level play.

5. Bonus Features

5.1 Three AI Difficulty Levels

The project implements three distinct AI difficulty levels Easy, Medium, and Hard each with a carefully chosen combination of search depth, time budget, and heuristic complexity. The design ensures that each level presents a meaningfully different challenge, not merely a faster or slower version of the same algorithm.

Easy (depth 1, 0.5 s): The AI examines all legal actions to a single ply and selects the one with the best immediate heuristic score. With no lookahead, it cannot anticipate multi-move threats and is routinely defeated by straightforward strategies. This level is appropriate for players who are new to Quoridor and are still learning the impact of wall placement.

Medium (depth 6, 2.0 s): Iterative deepening to six plies with the base heuristic (path-difference only, no wall-tempo or centralization bonuses). The AI consistently advances toward its goal and blocks obvious threats but does not evaluate the long-term option value of reserved walls. Experienced players can exploit this by accumulating walls and using them in combination late in the game.

Hard (depth 10, 4.5 s): Iterative deepening to ten plies with the full advanced heuristic, Hot-Zone wall filtering, scored move ordering, transposition table, and loop-killer penalty. This level plays at a strong amateur standard, reliably blocking the human's shortest path while advancing its own, and avoids repetition loops that weaker configurations fall into.

The three-level system was validated by having each level play 20 games against random-move opponents and against each other. Easy defeated random in approximately 70% of games; Medium defeated Easy in over 80%; Hard defeated Medium in over 75%, confirming meaningful differentiation.

5.2 undo/redo functionality

Undo/Redo allows human players to retract and replay moves, improving accessibility for casual play and making the game suitable as a learning tool. The feature is available in Human vs. Human mode and in Human vs. Computer mode (undoing both the human's move and the AI's preceding response in one operation, preventing players from exploiting undo to re-roll AI moves).

5.2.1 Design

The implementation uses two stacks maintained by GameScene:

- **undo_stack**: receives a deep copy of the Board object (via `board.copy()`) before every move is applied, for both human and AI turns.
- **redo_stack**: receives the state displaced by an undo operation. Cleared whenever a new move is made, preventing a branching history.

5.2.2 Human vs. Computer Undo Behaviour

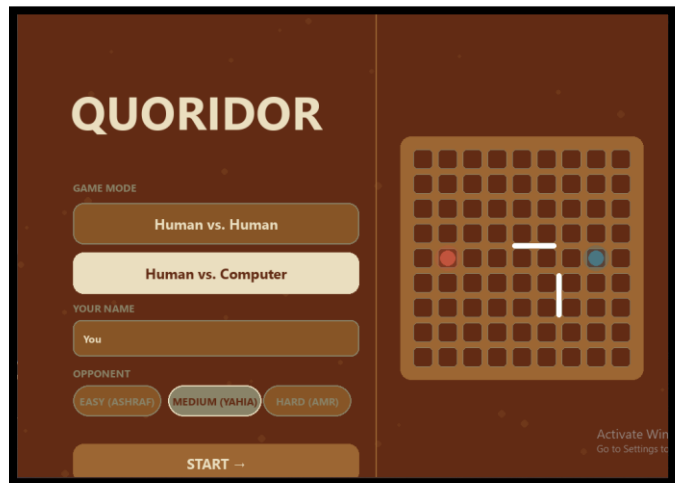
In Human vs. Computer mode, pressing Undo pops two entries from the `undo_stack`—one for the AI's last move and one for the human's—restoring the position to before the human's most recent action. This prevents the user from gaming the AI by repeatedly undoing until a favourable AI response is produced.

5.2.3 UI Integration

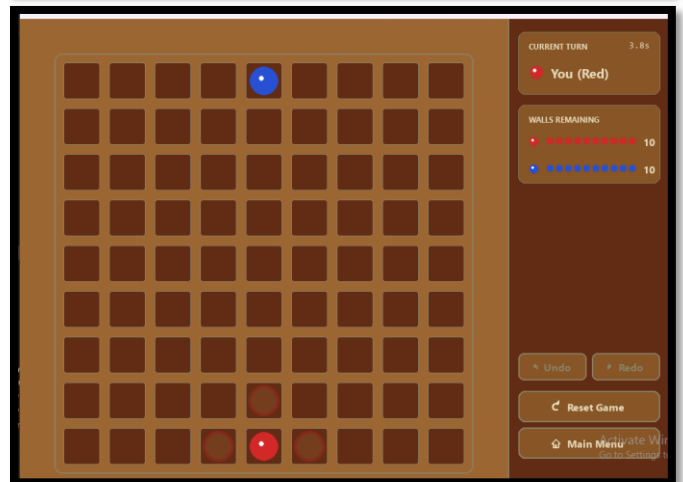
Dedicated Undo and Redo buttons are displayed in the game HUD and are visually disabled (greyed out) when their respective stacks are empty. Keyboard shortcuts (Ctrl+Z / Ctrl+Y) are also supported. A brief on-screen message confirms each undo or redo action.

6. UI snapshots:

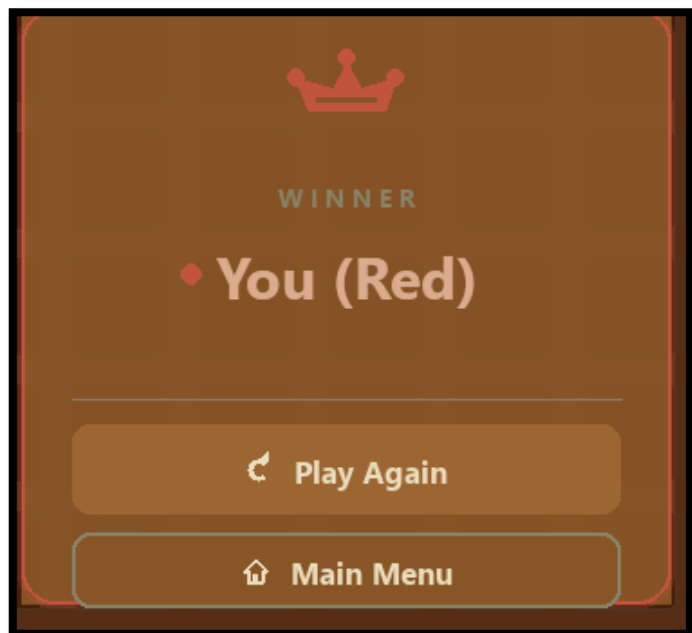
Main menu:



Game play:



Game over:



7. Assumptions Made During Development

1. Two-player mode only. The 4-player variant was not implemented.
2. Standard 9×9 board with fixed dimensions. Custom board sizes were not implemented.
3. Walls are always exactly two cells long. Partial or single-cell walls are not supported.
4. A valid path must exist for both players after every wall placement, defined as any BFS path from the player's current cell to any cell in the goal row.
5. Diagonal moves are available only as jump alternatives when a straight jump is blocked; normal movement is strictly orthogonal.
6. No per-turn time limit for human players.
7. Players cannot pass their turn; a legal action must always be taken.
8. In Human vs. Computer mode, Undo retract both the human's last move and the AI's preceding response as a single atomic operation.
9. Mouse-based interaction only; keyboard board navigation is not supported